CBT Frontend Implementation Guide

Base URL

`https://assess-ai-b2f175ea4c07.herokuapp.com/api/v1/question-service`

All endpoints return an envelope:

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 50 },
  "errors": null
}
```

Auth: `Authorization: Bearer <jwt-token>` on all requests.

---

1. Question Bank (Manage Questions)

Create a question

```
POST /questions
Content-Type: application/json

{
  "text": "What is 2+2?",
  "question_type": "multiple_choice",
  "options": ["3", "4", "5", "6"],
  "correct_answer": "4",
  "difficulty": "easy",
  "bloom_taxonomy": "remember",
  "tags": ["math", "basics"],
  "explanation": "2+2 equals 4",
  "course_id": "uuid",
  "module_id": "uuid",
```

```
  "topic_id": "uuid"
}
```

List/search questions

```
GET /questions?
page=1&limit=20&question_type=multiple_choice&difficulty=easy&tag=math&sea
rch=algebra
```

Get question detail

```
GET /questions/{id}
```

Update question (auto-versioning)

```
PATCH /questions/{id}
{ "text": "updated text", ... }
```

Get version history

```
GET /questions/{id}/versions
```

Manage tags

```
POST /questions/tags     { "name": "Algebra", "color": "#FF5733" }
GET  /questions/tags
```

---

2. Assessment Builder (Create Exams)

Create assessment

```
POST /assessments
{
  "title": "Midterm Exam - Mathematics",
  "description": "Covers chapters 1-5",
  "time_limit_minutes": 90,
  "shuffle_questions": true,
  "shuffle_options": true,
  "negative_marking": false,
```

```
  "passing_score_percent": 40,
  "max_attempts": 1,
  "available_from": "2026-07-20T00:00:00Z",
  "available_to": "2026-07-25T23:59:59Z",
  "grading_mode": "instant",
  "late_penalty_percent": 10,
  "instructions": "Read each question carefully. No calculators."
}
```

List assessments

```
GET /assessments?page=1&limit=20&status=published
```

Get assessment detail (with sections + items)

```
GET /assessments/{id}
```

Update / Delete / Publish / Archive

```
PATCH  /assessments/{id}       { "title": "updated" }
DELETE /assessments/{id}
POST   /assessments/{id}/publish
POST   /assessments/{id}/archive
```

Manage sections

```
POST   /assessments/{id}/sections    { "title": "Section A",
"order_index": 1 }
DELETE /assessments/{id}/sections/{section_id}
```

Add questions to assessment

```
POST /assessments/{id}/questions
{ "question_id": "uuid", "section_id": "uuid", "points": 5,
"negative_points": 1 }
```

Reorder questions

```
PATCH /assessments/{id}/reorder
{ "item_ids": ["uuid1", "uuid2", "uuid3"] }
```

---

3. Assessment Delivery (Student Exam Flow)

Step 1: Start exam

```
POST /assessments/{id}/delivery/start

Response:
{
  "submission_id": "uuid",
  "timer_expires_at": "2026-07-17T22:00:00Z",
  "time_limit_seconds": 5400,
  "questions": [
    {
      "item_id": "uuid",
      "question_id": "uuid",
      "type": "multiple_choice",
      "text": "What is 2+2?",
      "options": ["3", "4", "5", "6"],     ← correct_answer EXCLUDED
      "points": 5,
      "order_index": 1,
      "section_title": "Section A",
      "is_required": true
    }
  ],
  "assessment_title": "Midterm Exam",
  "instructions": "Read carefully...",
  "attempt_number": 1
}
```

Frontend action: Start timer countdown. Render questions in order. If `shuffle_options=true`, options are already shuffled server-side.

Step 2: Auto-save progress (every 30s)

```
POST /assessments/{id}/delivery/save
{
  "submission_id": "uuid",
  "responses": [
    { "question_id": "uuid", "answer_json": { "selected": "4" } },
    { "question_id": "uuid", "answer_json": { "text": "my answer" } }
```

```
  ]
}
```

Idempotent — safe to call repeatedly.

Step 3: Resume on reconnect

```
GET /assessments/{id}/delivery/resume?submission_id=uuid

Response:
{
  "submission_id": "uuid",
  "status": "in_progress",
  "seconds_remaining": 3200,
  "questions": [ ... ],          ← same as start
  "saved_answers": {
    "question_id_1": { "selected": "4" },
    "question_id_2": { "text": "my answer" }
  }
}
```

Step 4: Get server timer

```
GET /assessments/{id}/delivery/timer?submission_id=uuid

Response:
{
  "submission_id": "uuid",
  "seconds_remaining": 2900,
  "is_expired": false,
  "timer_expires_at": "2026-07-17T22:00:00Z"
}
```

Step 5: Submit exam

```
POST /assessments/{id}/delivery/submit
{
  "submission_id": "uuid",
  "responses": [
    { "question_id": "uuid", "answer_json": { "selected": "4" } }
  ]
```

```
}

Response (instant auto-grade for objective types):
{
  "submission_id": "uuid",
  "status": "submitted",
  "total_score": 35,
  "total_possible": 50,
  "graded_count": 8,
  "ungraded_count": 2,
  "is_late": false,
  "late_penalty": 0,
  "timer_expired": false
}
```

- MC/TF/FillBlank/MSQ/Matching → graded instantly, score included
- Short answer/Essay/Case study → ungraded, needs LLM grading step

Step 6: Proctoring events (send periodically)

```
POST /assessments/{id}/delivery/proctoring
{
  "submission_id": "uuid",
  "events": [
    { "event_type": "tab_switch", "details": { "count": 3 } },
    { "event_type": "window_blur", "details": { "duration_ms": 5000 } },
    { "event_type": "copy_paste", "details": { "field": "textarea_1" } },
    { "event_type": "idle", "details": { "duration_ms": 120000 } }
  ]
}
```

---

4. Submissions (Instructor View)

List submissions

```
GET /submissions?
page=1&limit=20&assessment_id=uuid&student_id=uuid&status=submitted
```

View submission detail (answers + proctoring events)

```
GET /submissions/{id}

Response includes per-question responses with auto-grade results +
proctoring events log.
```

Override score

```
POST /submissions/{id}/override
{
  "question_id": "uuid",
  "override_score": 4,
  "reason": "Partial credit for showing work"
}
```

---

5. Grading (Subjective Questions)

Trigger LLM grading

```
POST /grading/grade/{submission_id}
→ 202 Accepted
```

Get grading status

```
GET /grading/result/{submission_id}
```

Preview LLM grading (without saving)

```
POST /grading/preview
{
  "question_id": "uuid",
  "student_answer": "Newton's laws state...",
  "rubric_id": "uuid"
}
```

Create rubric

```
POST /grading/rubrics
{
  "question_id": "uuid",
```

```
  "criteria": [
    { "name": "Correctness", "max_points": 5, "description": "Answer is
factually correct" },
    { "name": "Explanation", "max_points": 3, "description": "Shows
understanding" }
  ],
  "total_points": 8
}
```

Get rubric

```
GET /grading/rubrics/{question_id}
```

List pending grading

```
GET /grading/pending?page=1&limit=20
```

---

6. Question Generation (Optional)

```
POST /generate       - LLM-generated questions from content
POST /generate/offline - NLTK rule-based (free, no API cost)
```

Frontend Architecture Suggestion

```
Pages:
  /instructor/assessments          → List, create, edit assessments
  /instructor/assessments/{id}     → Builder: sections + questions
  /instructor/questions            → Question bank CRUD
  /instructor/submissions          → List submissions
  /instructor/submissions/{id}     → View answers + trigger grading +
override
  /student/exams                   → Available/past exams
  /student/exams/{id}              → Exam player (timer + questions +
auto-save)
  /student/exams/{id}/results      → Score + feedback

Core Flow — Exam Player:
  1. On mount → POST .../delivery/start → get questions + timer
  2. Start countdown (client-side, sync with GET .../timer every 60s)
```

```
  3. Every 30s → POST .../delivery/save (auto-save)
  4. On tab switch/blur → POST .../delivery/proctoring
  5. On submit → POST .../delivery/submit
  6. If disconnect → GET .../delivery/resume on return

Question types rendering:
  multiple_choice  → radio buttons
  true_false       → radio (True/False)
  fill_blank       → text input (single/multiple)
  multiple_select  → checkboxes
  matching         → two-column drag-and-drop or dropdowns
  short_answer     → textarea (50-150 words)
  essay            → textarea (200-500 words)
  case_study       → scenario text + sub-questions below
```
